

PERSONAL PROJECT

AWS 자동 배포 마법사

개인 프로젝트 기획서

GitHub 주소 하나로 컨테이너가 뜨고 HTTPS 주소가 나오는 것.
거기서 시작해 필요할 때마다 한 겹씩 쌓아 올린다.

성격	1인 개인 프로젝트 · 마감 없음 · 배포 학습 병행
작성일	2026년 9월 17일
핵심 스택	Python(FastAPI) · boto3 · AWS ECS Express Mode · LLM

01 무엇을 만드는가

GitHub 주소와 AWS 임시 권한, 딱 두 가지만 받아서 배포하고 관리한다.

AI가 Dockerfile을 쓰고, 빌드해서, AWS에 올리고, 접속 가능한 HTTPS 주소를 돌려준다. 실패하면 되돌리고, 원인을 한국어로 설명한 뒤 어떻게 할지 묻는다.

마감이 없으므로 “언제까지 무엇을”이 아니라 “**어느 순서로 쌓을 것인가**”가 이 문서의 주제다. 각 마일스톤은 그 자체로 완결되며, 어디서 멈춰도 돌아가는 상태를 유지한다.

이 프로젝트로 얻는 것

- **실물** — 남에게 보여줄 수 있는, 실제로 작동하는 배포 도구
- **AWS 실전 지식** — 컨테이너·IAM·네트워크·로드밸런서를 문서가 아니라 손으로 익힌다
- **에이전트 설계 경험** — LLM으로 코드를 읽고 판단하고 고치는 루프를 직접 만든다

하지 않기로 한 것

범위를 좁히는 것이 아니라 **순서를 뒤로 미루는** 것이다. 08장에 확장 경로가 있다.

- 워커·배치·크론 — Express Mode의 대상이 아니다 (02장)
- 모바일 앱 빌드 — 웹과 파이프라인이 완전히 다르다
- 딥러닝 모델 — 학습 데이터가 쌓인 뒤의 일이다

02 왜 이 설계인가

ECS Express Mode를 실행 엔진으로 쓴다

공식 문서로 확인한 사실 — Express Mode는 ECS 클러스터, 태스크 정의, ALB(HTTPS 리스너·타겟그룹), 보안 그룹, IAM 역할, CloudWatch 로그, ACM 인증서를 자동 생성한다. 같은 VPC 내 최대 25개 서비스가 ALB 1개를 공유하고, 초과 시 자동 추가·해제된다.

VPC·로드밸런서·인증서를 직접 조립하지 않으므로 혼자서도 완주 가능한 난이도로 내려온다. 그리고 결과물이 고객 계정의 정식 ECS 리소스라 나중에 Terraform으로 내보낼 수도 있다.

그리고 여기에 이 프로젝트의 존재 이유가 있다. Express Mode는 컨테이너 이미지만 입력받고 소스코드는 받지 않는다. 즉 Dockerfile을 쓰고 빌드 파이프라인을 짜는 일은 여전히 사람 몫이다. “소스코드 → 이미지” 구간이 AWS가 비워둔 자리이고, 내가 메우는 자리다.

Express Mode가 다루지 않는 것

유형	왜 대상이 아닌가
백그라운드 워커	ALB 뒤 HTTP 서비스 구조가 전제. 응답할 웹 서버가 없으면 헬스체크 실패로 재시작이 반복된다
배치·크론	끝나면 종료돼야 하는데 Express Mode는 서비스를 계속 살려둔다
상태 저장 앱	stateless 전제. 컨테이너 재시작 시 로컬 파일이 사라진다
데이터베이스	생성 리소스 목록에 DB가 없다. RDS는 직접 만들어야 한다 (M6)
사이드카·FireLens	커스텀 태스크 정의가 필요하다

대안 경로 — 워커는 로드밸런서 없는 ECS 서비스, 크론은 EventBridge Scheduler + ECS RunTask, 배치는 AWS Batch.

LLM은 “빈칸 채우기”로만 쓴다

작업	방식	이유
언어·런타임 판별	규칙	매니페스트 파일 존재 여부. 결정적이고 빠르고 공짜
Dockerfile 생성	템플릿 + LLM	매번 자유 생성하면 어제 되던 게 오늘 안 된다. 템플릿을 두고 빌드 명령·시작 명령·포트·버전만 묻는다. 출력은 JSON으로 고정
빌드 실패 수정	LLM	로그를 읽고 고치는 건 LLM이 압도적. 최대 2회로 제한
에러 해설	LLM	이 도구의 실질적 가치

레포 전체를 보내지 않는다. 디렉터리 트리(2~3단계), 매니페스트, 진입점 파일 1~2개만. .env·.git·node_modules·키 파일은 제외 필터 필수.

언어 지원 — 정식과 실험 두 층

LLM은 어떤 언어의 Dockerfile이든 쓴다. 비용은 생성이 아니라 “되는지 확인하는 작업”에 있다. 언어마다 빌드 실패 양상이 달라 자가수정 프롬프트를 따로 튜닝해야 하기 때문이다. 그래서 언어를 막지 않고 두 층으로 나눈다.

층	대상	동작
정식 지원	Node.js · Python	검증된 템플릿 + LLM 빈칸 채우기 + 튜닝된 자가수정(2회)
실험 지원	그 외 전부	LLM 순수 생성으로 일단 시도. 화면에 “실험 지원”을 표시하고, 실패해도 정상으로 처리

매니페스트 감지

└─ **package.json / requirements.txt** → 정식 템플릿 경로

└─ **그 외** (pom.xml, go.mod, composer.json, Gemfile ...) → LLM 순수 생성 (실험) → 성공 시 결과를 템플릿 후보로 저장

승격 기준 — 같은 언어로 **5개 레포 중 4개 이상 성공**하면 템플릿을 만들어 정식 지원으로 올린다. “언제 추가할까”를 매번 고민하지 않아도 실험 로그가 알려준다. 실패 로그 역시 그 언어 템플릿의 재료가 된다.

실험 경로의 안전장치 둘 — ① 자가수정을 **1회로 제한**한다. 모르는 언어에서 계속 고치겠다고 달려들면 토큰과 빌드 시간이 같이 샌다. ② **빌드 타임아웃을 넉넉히** 잡는다. Java·Rust는 Node 기준 타임아웃이면 정상 빌드도 실패로 처리된다.

언어별로 예상되는 함정

정식 지원으로 승격할 때 확인할 목록.

언어	그 언어만의 함정
Java	JDK 버전, Maven/Gradle 선택, 멀티스테이지를 안 쓰면 이미지가 수백 MB. 빌드가 오래 걸려 타임아웃이 달라지고, 메모리를 많이 써서 기본 사양도 달라진다
Go	CGO를 끄고 정적 빌드해야 가벼운 베이스 이미지에서 뜬다. 안 그러면 실행 시 라이브러리를 못 찾고 죽는다
PHP	웹서버가 따로 필요 (php-fpm + nginx). 컨테이너 하나에 프로세스 하나라는 원칙과 충돌한다
Ruby	네이티브 확장 gem 컴파일에 시스템 패키지가 빠져 실패하는 경우가 많다

03 권한 — 사용자의 GitHub와 AWS를 받는다

이 도구는 **사용자의 GitHub에서 코드를 읽고, 사용자의 AWS 계정에 리소스를 만든다**. 둘 다 강한 권한이므로, 어떻게 받고 어떻게 보관하는지가 설계의 일부다.

GitHub — 단계적으로 올린다

방식	특징	언제
권한 없음	public 레포만 clone. 토큰이 아예 필요 없다	M3까지 — 개발 초기엔 이걸로 충분하다
PAT 입력	사용자가 개인 액세스 토큰을 붙여넣는다. 구현이 가장 쉽지만 토큰을 우리가 보관하게 된다	private 레포가 필요해지는 시점
GitHub App	설치형. 레포 단위로 권한을 고르고 , 발급 토큰의 수명이 짧다(약 1시간). 사용자가 언제든지 앱을 제거해 회수할 수 있다	M6 — 남이 쓰게 할 때의 정답

요청할 권한은 최소로. 처음에는 `contents:read`와 `metadata:read`만 있으면 된다. 코드 수정 PR을 만드는 단계(M9 이후)에 가서야 `pull_requests:write`를 더한다.

AWS — 키를 받지 않고 역할을 위임받는다

액세스 키를 입력받는 방식은 쓰지 않는다. 장기 자격증명을 우리가 보관하게 되고, 유출되면 회수가 늦다. 사용자가 키를 직접 폐기하기 전까지 계속 유효하다.

순서	동작	비고
1	사용자가 우리 화면의 링크 클릭	CloudFormation 스택 생성 화면으로 이동
2	자신의 AWS 콘솔에서 스택 생성	필요 권한을 가진 IAM 역할이 만들어진다
3	생성된 역할 ARN 을 우리 화면에 등록	External ID 를 함께 써서 제3자가 우리 신원을 흉내내 접근하는 것을 막는다
4	우리 서버가 <code>sts:AssumeRole</code> 로 임시 자격증명 획득	수명이 짧고, 사용자가 스택을 지우면 즉시 끊긴다

임시인 것은 키이고, 권한은 유지된다. 사용자는 역할을 한 번 만들어 두고, 우리는 작업할 때마다 그 역할로 **1시간짜리 키를 새로 받는다**. 키가 새어 나가도 한 시간이면 무효가 되고, 사용자가 스택을 지우는 순간부터는 아무것도 할 수 없다. 관리는 지속되지만 끊을 권한은 항상 사용자에게 있다.

역할에 담은 권한 — 배포와 관리에 필요한 것만

서비스	범위	용도
ECS	서비스 생성·수정·조회	배포, 롤백
ECR	리포지토리 생성, 이미지 푸시	빌드 결과 업로드
CloudWatch Logs	읽기	문제 이력, 원인 해설
RDS	생성·조회	M7에서 추가

우리가 만든 리소스에는 **managed-by: deploy-wizard** 태그를 붙이고, 수정·삭제 권한을 그 태그가 붙은 리소스로 한정한다. 사용자 계정의 다른 자원은 건드릴 수조차 없게 역할 정책 단계에서 막는다.

권한 없이 되는 것도 있다 — 헬스체크와 갤러리 스크린샷은 이미 공개된 HTTPS 주소를 호출하는 것이라 AWS 권한이 필요 없다.

코드 변경 감지 — 단계적으로 붙인다

GitHub 주소만으로는 코드가 바뀐 것을 알 수 없다. 하나를 고르는 것이 아니라 순서대로 쌓는다.

시점	방식	내용
M3~M5	수동 버튼	대시보드의 [다시 배포]. 구현이 거의 없고 초기엔 이걸로 충분하다. 배포가 제대로 되는 것이 먼저다
그다음	폴링 — <code>git ls-remote</code>	5분마다 대상 브랜치의 커밋 해시 한 줄만 받아 저장값과 비교한다. REST API가 아니라 git 프로토콜이라 API 호출 한도와 무관 하고 레포 전체를 받지 않는다
M6	웹훅 + 폴링 백업	GitHub App 설치 시 웹훅을 함께 켜다. 푸시 즉시 반영, private 레포 지원. 웹훅은 가끔 유실되므로 폴링을 1시간 간격으로 남겨 놓친 것을 잡는다

배포 설정 — 사용자가 고른다

설정	기본값	동작
대상 브랜치	레포의 기본 브랜치	드롭다운으로 선택. 목록은 <code>git ls-remote --heads</code> 로, 기본 브랜치는 <code>git ls-remote --symref ... HEAD</code> 로 가져온다 — 둘 다 API 없이 된다. 이 브랜치에 들어온 푸시만 재배포한다
자동 재배포	켜짐	끄면 푸시가 와도 배포하지 않고 “새 커밋이 있습니다”만 표시한다. 푸시는 자주 하되 배포는 원할 때만 하고 싶은 사용자를 위한 선택

상황	동작
대상 브랜치를 바꿈	“ 지금 이 브랜치로 다시 배포할까요? ”로 확인한다. 설정 변경이 곧바로 배포로 이어지면 안 된다
여러 브랜치를 배포하고 싶음	하나의 배포 = 하나의 브랜치 . main은 운영, dev는 테스트처럼 배포를 따로 만든다. ALB를 25개까지 공유하므로 추가 비용 부담이 작다
대상 브랜치가 삭제됨	자동 재배포를 멈추고 알린다. 이미 떠 있는 서비스는 그대로 둔다
짧은 시간에 연속 푸시	30초 정도 기다렸다가 마지막 커밋 하나만 배포한다. 진행 중인 배포가 있으면 끝난 뒤 최신 것만 이어서

푸시는 사용자의 의사표시이므로 자동 재배포는 “수정은 묻는다” 원칙과 충돌하지 않는다. 재배포가 실패하면 아래 정책대로 롤백 후 묻는다.

웹훅 서명 검증은 필수다. GitHub은 요청마다 X-Hub-Signature-256 헤더에 HMAC-SHA256 서명을 넣는다. 검증하지 않으면 누구나 가짜 푸시 이벤트로 배포를 일으킬 수 있다. 로컬 개발 중에는 우리 서버에 공개 주소가 없으므로 **smee.io** 같은 중계로 웹훅을 넘겨받는다.

자동으로 하는 것은 하나 — 되돌리기

원칙은 단순하다. 일단 정상 작동으로 돌려놓는 것만 자동으로 하고, 새로 만들거나 바꾸거나 지우는 것은 전부 사용자에게 묻는다.

상황	자동으로	사용자에게 묻기
배포 실패 직전 정상 버전 있음	직전 버전으로 롤백	원인 해설과 함께 [수정안 보기] [다시 배포] [그대로 두기]

첫 배포 실패	— 돌아갈 버전이 없다	원인 해설 + 만들어진 리소스를 정리할지
운영 중 장애	ECS가 태스크를 재시작 (AWS 기본 동작)	원인 해설 + 조치안
롤백 자체 실패	멈춘다	즉시 알림
설정·코드 수정	하지 않는다	항상 확인 — 수정안을 보여주고 승인받는다
리소스 삭제	하지 않는다	항상 확인
Dockerfile 자가수정	빌드 단계에서 최대 2회	— 우리가 생성한 파일이라 사용자 코드를 건드리지 않는다

묻는 방식은 선택지로. “어떻게 할까요?” 같은 열린 질문이 아니라 버튼 두세 개를 준다. 사용자가 판단하기 쉬워야 실제로 답을 한다.

M1에서 확인할 것 — ECS에는 배포가 실패하면 이전 버전으로 되돌리는 배포 서킷 브레이커가 있고, Express Mode는 카나리 배포와 배포 실패 감지 알람을 함께 만든다. **이 조합이 롤백을 대신해 주는지 확인한다.** 된다면 롤백을 직접 구현할 필요 없이 결과를 받아 사용자에게 알리기만 하면 된다.

보관 원칙

- **평문 저장 금지.** 토큰·ARN·External ID는 Secrets Manager에 두거나 KMS로 암호화해 저장한다
- **로그에 찍히지 않게.** 토큰이 예외 메시지나 디버그 출력에 섞여 나가는 사고가 가장 흔하다
- **회수 가능하게.** GitHub 앱 제거, CloudFormation 스택 삭제 — 사용자가 우리를 거치지 않고 권한을 끊을 수 있어야 한다
- **레포 내용은 통째로 LLM에 보내지 않는다.** 디렉터리 트리·매니페스트·진입점 파일만. .env·키 파일 제외 필터는 필수다 (02장)

설계 제약 — M1부터 지킬 것
모든 AWS 호출이 **세션을 주입받는 구조**여야 한다. 개발 중에는 내 계정 세션을 넣고, M6에서 AssumeRole 세션으로 갈아끼운다.

```
def deploy(session, repo_url): ecs = session.client('ecs') ← 이렇게
ecs = boto3.client('ecs') ← 전역 클라이언트로 두면 M6에서 전부 다시 쓴다
```

04 마일스톤 — 날씨가 아니라 순서

개인 프로젝트는 중단과 재개가 반복된다. 그래서 원칙은 하나다. **어디서 멈춰도 돌아가는 상태로 쌓는다.** 각 마일스톤은 “완료 기준”이 명확하고, 그 시점에서 손을 떼도 무언가는 작동한다.

M0 · 준비

콘솔에서 손으로 한 번 배포해 본다

AWS 계정 생성, **예산 알림 설정**, 콘솔에서 Express Mode로 공개 이미지(nginx) 하나 배포.

완료 기준 — 브라우저에서 주소가 열린다 배우는 것: Express Mode가 무엇을 만드는지 눈으로 확인

M1 · 첫 성공 ★★★★★☆

코드로 컨테이너 하나 띄우기

boto3로 Express Mode를 호출해 M0과 같은 배포를 **코드로** 재현. 상태 폴링으로 진행 상황을 콘솔에 출력.

완료 기준 — 스크립트 한 번 실행에 주소가 나온다 배우는 것: boto3, IAM 권한, AWS 상태값 흐름 · **가장 중요한 마일스톤. 여기서 되면 나머지는 앞에 파이프를 붙이는 일**

M2 · 빌드 ★★★★★☆

내가 만든 이미지를 올려서 띄우기

로컬 Dockerfile로 이미지 빌드 → ECR 리포지토리 생성 → 푸시 → 그 이미지로 배포.

완료 기준 — 내 코드가 AWS에서 돈다 배우는 것: 컨테이너 레지스트리, 이미지 태깅 · **arm64/amd64 합정을 여기서 만난다 (06장)**

M3 · 핵심 ★★★★★☆

GitHub 주소만 넣으면 배포된다

레포 클론 → 언어 판별(규칙) → Dockerfile 생성(템플릿+LLM) → 빌드 → 푸시 → 배포. CLI로 동작하면 충분하다. **모르는 언어는 실험 경로로 보낸다(02장).**

완료 기준 — 주소 하나로 끝까지 간다 배우는 것: LLM 구조화 출력, 프롬프트 설계 · **여기까지가 “보여줄 수 있는 것”의 최소 단위**

M4 · 화면 ★★★★★☆

웹 UI와 실시간 진행률

입력 화면, 단계별 진행률, 터미널형 로그 출력, 완료 시 클릭 가능한 링크. 그리고 **내가 배포한 서비스 목록**과 각각의 현재 상태(정상·장애·중지)를 한 화면에서 본다.

완료 기준 — 브라우저에서 처음부터 끝까지, 배포한 것들이 목록에 남는다 배우는 것: 비동기 작업 상태 관리, 폴링/스트리밍

M5 · 신뢰 ★★★★★☆

실패해도 죽지 않는다

각 단계 실패 처리, **한국어 에러 해설**, 빌드 실패 시 자가수정 2회, 중단 지점 복구. **문제 이력을 서비스별로 모아 본다** — 언제 무엇이 왜 죽었는지가 한 곳에 쌓인다.

완료 기준 — 일부러 망가뜨린 레포에서 원인이 한국어로 나온다 배우는 것: 에이전트 루프 설계, 종료 조건 · **여기부터 “도구”가 된다**

M6 · 권한 ★★★★★☆

사용자의 GitHub와 AWS를 받아서 쓰기

GitHub App 설치로 레포 읽기 권한, CloudFormation 링크 + **sts:AssumeRole**로 AWS 권한. 토큰은 암호화 보관, 로그에서 마스킹. (03장)

완료 기준 — 내 계정이 아닌 곳에, private 레포로 배포된다 배우는 것: OAuth·GitHub App 설치 흐름, STS 임시 자격증명, 비밀 관리
· M1의 세션 주입 구조가 여기서 값을 한다

M7 · 확장 ★★★★★★

데이터베이스까지 자동 생성

RDS 생성, 서버넷 그룹, **ECS→RDS 보안그룹 인바운드**, 비밀번호 관리, 접속정보 주입, 마이그레이션.

완료 기준 — DB 쓰는 앱이 무설정으로 뜬다 이 프로젝트에서 가장 어려운 구간. 급하지 않으니 천천히 가도 된다

M8 · 확장 ★★★★★☆

프론트엔드와 백엔드 동시 배포

백엔드 배포 → 주소 획득 → 그 주소를 넣고 프론트 빌드 → 배포. 순서 의존이 생긴다.

완료 기준 — 풀스택 레포가 통째로 뜬다

M9 · 확장

운영 비서 — 죽으면 알려주고 되돌린다

로그 감시(규칙 기반) → 이상 감지 → LLM 원인 해설 → 조치안 제시. **자동으로 하는 것은 롤백 하나**, 나머지는 선택지로 묻는다 (03장).

완료 기준 — 서비스를 죽였을 때 알림이 오고 복구된다

M10 · 확장

학습 모드

각 단계에서 “지금 무슨 일이 일어나는가”, 완료 후 “무엇이 왜 만들어졌나”, “직접 하려면 이 명령” 토글.

설명 텍스트는 미리 써둔 고정 텍스트로. 매번 생성하면 틀린 설명이 나간다

M11 · 확장

쇼케이스 갤러리 — 배포한 서비스를 알린다

배포 완료 후 “이 서비스를 갤러리에 공개할까요?” 기본은 비공개, 공개는 옵트인. 스크린샷 자동 캡처, 한 줄 소개, 기술 스택 태그.

완료 기준 — 남의 서비스가 목록에 뜨고 클릭하면 열린다 우리만 가능한 이유 둘: 태그가 자동으로 붙는다(배포할 때 이미 언어·프레임워크·DB를 분석했다) · 죽은 링크가 없다(헬스체크로 내려간 서비스를 자동으로 숨긴다)

갤러리 — 세 단계로 연다

유료 노출은 방문자가 생긴 뒤에야 팔린다. 광고부터 만들면 아무도 사지 않는다.

단계	무엇을	비고
1. 공개	배포 후 공개 여부를 묻는다. 커면 갤러리에 오른다	기본값은 비공개. 스크린샷·기술 스택은 이미 가진 데이터로 자동으로 채운다 — 사용자는 한 줄 소개만 쓰면 된다
2. 정화	목록에 신고 버튼을 둔다	무료 갤러리라도 아무거나 올라오면 안 된다. 신고 사유는 몇 개 항목으로 고르게 해서 처리 판단을 쉽게 한다

3. 수익

등록 서비스와 방문자가 쌓이면 **유료 상단 고정·카테고리 상단·주간 소개**

대기 시간 광고 지면과 묶어서 판다. 지면의 청중이 개발자로 특정된다는 것이 강점

신고 처리 — 혼자 감당할 수 있는 선에서

- **신고 N건이 쌓이면 자동으로 임시 숨김** 처리하고, 그 뒤에 내가 확인한다. 사람이 먼저 보고 판단하는 구조는 혼자서는 감당이 안 된다
- **신고 사유를 고르게 한다** — 사기·불법·성인물·작동하지 않음·기타. 자유 서술만 받으면 판단에 시간이 배로 든다
- **재공개 요청 창구**를 둔다. 오인 신고로 내려간 서비스가 항의할 곳이 없으면 그게 더 큰 문제가 된다
- **헬스체크와 신고는 다른 문제다** — 죽은 서비스는 자동으로 숨기면 되지만, 내용이 문제인 서비스는 사람이 봐야 한다

공개 갤러리를 여는 순간 운영 업무가 생긴다. 개인 프로젝트에서 이게 가장 과소평가되는 비용이다. **M11은 다른 마일스톤이 안정된 뒤에 연다.** 열어두고 방치하면 도구 전체의 신뢰가 같이 깎인다.

05 비용 관리 — 개인 프로젝트의 진짜 제약

마감이 없는 대신 **요금은 매달 나온다**. 띄워둔 리소스는 쓰지 않는 동안에도 과금되고, 프로젝트가 몇 달 이어지면 그 차이가 커진다. **비용 관리가 일정 관리보다 중요하다**.

항목	과금 방식	대응
ALB	Express Mode가 자동 생성. 떠 있는 동안 계속 과금	실습이 끝나면 서비스를 삭제한다. 25개 공유 구조라 여러 개 띄워도 ALB는 하나
RDS	인스턴스 시간당. 가장 비싸다	M6 작업 중에만 띄운다. 스냅샷 남기고 삭제하는 습관
Fargate	vCPU·메모리 × 실행 시간	최소 사양으로 실습
ECR	이미지 저장 용량	오래된 태그 정리
LLM API	토큰	같은 레포·커밋이면 분석 결과 캐싱 . 반복 테스트에서 비용이 크게 준다

지금 당장 해둘 것

- **AWS Budgets에 예산 알림 설정**. 금액을 넘으면 메일이 오게. 이것만 해두면 사고가 나도 하루 안에 안다
- **프리티어·학생 크레딧 확인**
- **정리 스크립트를 M1에서 같이 만든다**. 배포 코드를 짜면서 삭제 코드도 같이 짜두면, 실습 끝날 때마다 한 번 실행하면 된다. 나중에 만들면 안 만들게 된다

습관 하나 — 작업을 끝낼 때 “오늘 만든 리소스 다 지웠나”를 확인한다. 개인 프로젝트에서 요금이 새는 건 대부분 **실습하고 그냥 둔 것**이다.

06 막힐 지점과 대응

합정	증상	대응
아키텍처 불일치	M1/M2 맥에서 빌드한 이미지를 올리면 exec format error로 죽는다	처음부터 --platform linux/amd64를 박아둔다. 가장 흔한 사망 원인
IAM 권한	첫 배포까지 가는 길이 대부분 권한 오류. 에러가 원인을 알려주지 않는다	처음엔 넓게 열고 나중에 좁힌다. 최소 권한부터 하려다 이틀을 쓴다
ECR 토큰 만료	어제 되던 푸시가 오늘 안 된다 (12시간 만료)	매번 새로 받도록 코드에 넣는다
AI의 허구 API	Express Mode는 2025년 11월 신규 기능이라 존재하지 않는 파라미터를 지어낸다	이 부분만 공식 문서를 열어놓고 대조. RDS·ECR은 오래된 API라 해당 없음
ECS→RDS 접속	앱은 뚫는데 DB 연결이 안 된다 (M6)	보안그룹 인바운드를 ECS 보안그룹 기준으로 연다. 이 디버깅에 시간이 가장 많이 든다
순서 의존	프론트는 빌드할 때 API 주소가 박힌다 (M7)	백엔드 완료를 기다렸다 프론트를 빌드. 실패 시 어디서 멈췄는지 기록

작업 규칙

- 2시간 룰 — 막힌 걸 2시간 넘게 붙잡지 않는다. AWS 권한·네트워크 문제는 에러가 원인을 알려주지 않아 붙잡을수록 시간이 샌다. 접근을 바꾸거나 우회한다
- 막힌 건 기록한다 — 에러 메시지 원문을 그대로. 같은 에러는 반드시 다시 만난다
- CLAUDE.md를 둔다 — 세션이 바뀌면 AI는 규칙을 잊는다. 상태값·네이밍·폴더 구조를 적어둔다
- 담당 파트를 읽는다 — AI가 짠 코드가 돌아가는 것과 이해하는 것은 다르다. 이 프로젝트의 목적 절반은 배우는 것이다

07 나중에 서비스로 만든다면

지금은 개인 프로젝트지만, 구조상 서비스로 확장할 수 있다. 그때를 위한 메모.

항목	정리
포지셔닝	배포 도구가 아니라 운영 비서. 배포는 진입점이고 운영이 본진이다. “대신 해주면서, 어떻게 했는지 알려준다”
경쟁 구도	AWS 콘솔은 결정을 전가하고, Vercel은 영어권·프론트 중심. Vercel도 AWS 위에서 돌아간다 — 편의와 판단만으로 시장을 만들었다는 것이 이 방향의 증거
비용 구조	서버비는 사용자가 자기 AWS 계정에 직접 지불(M6 이후). 우리 원가는 토큰비뿐이고, 규칙 필터 → 저가 모델 분류 → 고성능 모델 분석의 3단 구조와 에러 시그니처 캐싱으로 상한을 둔다
자율성	자동은 되돌리기 하나. 만들고 바꾸고 지우는 것은 전부 사용자 확인. 신뢰가 쌓이고 요청이 생기면 그때 항목별 자동 허용을 옵션으로 연다
노출 수익	갤러리를 공개(오픈트인) → 신고로 정화 → 유료 상단·광고 순으로 연다(M11). 지면의 청중이 개발자로 특정된다는 것이 강점이고, 등록 서비스가 쌓이기 전에는 팔리지 않는다
기술 자산	배포 실패 로그가 쌓이면 실패 원인 분류 모델의 학습 데이터가 된다. 대행하기 때문에 데이터가 손에서 만들어진다

08 다음에 할 일

1. AWS 계정 생성 + 예산 알림 설정
2. 콘솔에서 Express Mode로 nginx 이미지 배포 (M0)
3. 같은 배포를 boto3 코드로 재현 + 정리 스크립트 (M1)

3번이 되면 이 프로젝트는 사실상 시작된 것이다. 나머지 마일스톤은 전부 그 앞뒤에 파이프를 붙이는 일이다.

※ 검증 출처: Amazon ECS 개발자 안내서(Express Mode 개요·생성 리소스·모범 사례). Express Mode 관련 코드는 AI 생성물을 반드시 공식 문서와 대조할 것.